THE NATIONAL COLLEGE

(AUTONOMOUS)

BASAVANAGUDI, BENGALURU -560004

॥ ಶ್ರದ್ಧಾಹಿ ಪರಮಾ ಗತಿಃ ॥

PROJECT REPORT ON

"AI-Driven Cybersecurity Threat Intelligence Platform"

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF UNDERGRADUATE DEGREE

UNDER THE GUIDENCE OF

Prof. PAVITHRA D, MCA (Ph.D)

DEPARTMENT OF COMPUTER SCIENCE

DURING THE ACADEMIC YEAR 2024-2025

SUBMITTED BY

HARSHA KRISHNA FATTEPUR

U18EZ22S0140

THE NATIONAL COLLEGE

(AUTONOMOUS)

BASAVANAGUDI, BENGALURU - 560004

॥ ಶ್ರದ್ಧಾಹಿ ಪರಮಾ ಗತಿಃ ॥

Certificate

This to certify that the **PROJECT WORK** entitled **"AI-DRIVEN CYBERSECURITY THREAT INTELLIGENCE PLATFORM"** is a bonafide work carried out by **HARSHA KRISHNA FATTEPUR(U18EZ22S0140)** in the partial fulfilment for the award of Sixth Semester **BACHELOR OF COMPUTER APPLICATIONS** of **THE NATIONAL COLLEGE**, **Basavanagudi**, **Bengaluru-04** during the year **2024-2025.**

Prof. Ravi Hegde
Head of the Department
Department of Computer Science

Prof. Pavithra D
Teacher in Charge
Department of Computer Science

Examiners:

1. _____

2. _____

ACKNOWLEDGEMENT

I take this opportunity to express my profound sense of gratitude and respect to all those who helped me throughout the duration of my project. The beatitude, bliss and euphoria that accompany the successful completion of any task would not be completed without the expression of appreciation of simple gratitude to the organization and people who made it possible.

I am highly indebted to The National College for providing me with such a intellectual platform and support to carry out my project work successfully.

I wish to express my deep sense of gratitude to, Mrs. Pavithra D, Assistant Professor, The National College for his able guidance and useful suggestions, which helped me in completing the project work, in time.

Words are inadequate in offering thanks to all my teachers for their constant guidance and support throughout this project work.

Finally, yet importantly, we would like to express my heartfelt thanks to my beloved parents for their blessings, my friends for their help and wishes for the successful completion of this project.

BY:

HARSHA KRISHNA FATTEPUR

Department of Computer Science
The National College, Basavanagudi, Bengaluru-560004

TABLE OF CONTENTS

Sl. No.	INDEX	Pg. No
1	Abstract	1
2	Introduction	2 - 3
3	Existing System and Proposed System.	4
4	Software Requirement Specifications.	5 - 6
5	System Analysis	7 - 8
6	System Design	9 - 10
7	Database Design (Table Structure and ER diagram)	11 - 13
8	Implementation	14 - 15
9	Testing	16 – 17
10	Future Enhancement and Conclusion	18
11	Bibliography	19
12	Appendices	20 - 64

1. ABSTRACT

In today's digital landscape, cybersecurity threats are evolving rapidly, posing significant risks to organizations and individuals. Traditional security measures often fall short in identifying sophisticated attacks, necessitating a more proactive approach. This project presents an **AI-Driven Cybersecurity Threat Intelligence Platform** that leverages artificial intelligence, machine learning, and automation to detect, analyze, and mitigate cyber threats in real time.

The platform addresses key limitations of conventional security systems, such as delayed response times, high false positives, and limited adaptability to emerging attack vectors. By incorporating behavioural analysis and deep learning techniques, it enhances threat detection accuracy and automates response mechanisms, reducing human intervention while ensuring robust cybersecurity measures.

The study outlines the system architecture, methodology, and implementation strategies, detailing the integration of AI models for predictive analytics and automated incident response. Future enhancements include improved threat prediction capabilities, blockchain integration, and enhanced scalability. This AI-driven approach strengthens cybersecurity resilience and offers actionable intelligence for security professionals in an increasingly hostile digital world.

2. INTRODUCTION

2.1 General Information

In today's rapidly evolving digital landscape, cyber threats pose a significant risk to organizations, governments, and individuals. Traditional security measures often struggle to keep up with the ever-changing nature of cyberattacks. An AI-Driven Cybersecurity Threat Intelligence Platform harnesses the power of artificial intelligence to detect, analyze, and mitigate security threats proactively. By leveraging machine learning and automation, this platform enhances threat intelligence capabilities and helps in real-time detection of malicious activities.

2.2 Statement of the Problem

The rise in cyber threats, including malware, phishing, ransomware, and advanced persistent threats (APTs), has revealed the limitations of conventional security solutions. Reactive approaches lead to delayed responses, often resulting in data breaches and financial losses. Current threat detection systems lack adaptability to emerging attack vectors, requiring a more intelligent, predictive, and automated approach to cybersecurity.

2.3 Objective of the Study

The main objectives of this project are:

•To develop an AI-powered threat intelligence platform capable of real-time cybersecurity threat detection and analysis.

•To improve the accuracy and speed of threat identification using machine learning algorithms.

•To automate threat mitigation processes, reducing human intervention while ensuring robust cybersecurity measures.

•To enhance cybersecurity resilience by continuously learning from evolving cyber threats.

2.4 Methodology

This study utilizes a combination of machine learning, deep learning, and natural language processing (NLP) techniques to analyze cybersecurity threats. Data is collected from various sources such as threat intelligence feeds, logs, and network traffic. AI algorithms are trained to recognize attack patterns and predict potential threats. The methodology also includes system implementation, testing, and performance evaluation of the platform.

2.5 Scope and Limitation of the Study

This platform is designed to handle multiple types of cyber threats and provide actionable insights for cybersecurity professionals. However, certain limitations exist:

- The accuracy of AI-driven detection relies on the quality and quantity of training data.

- While automation enhances threat response, human oversight remains crucial for complex cybersecurity incidents.

- The platform requires continuous updates to adapt to new and sophisticated cyber threats.

2.6 Hardware and Software Requirements

The implementation of this project requires specific hardware and software components:

Hardware Requirements:

- High-performance computing system with GPU acceleration for AI processing.

- Network infrastructure for data collection and real-time monitoring.

Software Requirements:

- Python-based frameworks for AI and machine learning (TensorFlow, PyTorch).

- Threat intelligence databases and cybersecurity APIs.

- Cloud-based deployment and security monitoring tools.

3. EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing System

Cybersecurity threat detection in traditional systems relies heavily on rule-based mechanisms, signature matching, and manual analysis. These systems use predefined threat patterns to detect malicious activities. However, they suffer from several limitations:

- Lack of adaptability: Traditional systems struggle to detect new and unknown threats, making them ineffective against zero-day attacks.

- Slow response time: Manual threat analysis increases detection delays, allowing attackers to exploit vulnerabilities before mitigation.

- High false positives: Conventional methods often misclassify legitimate network activities as potential threats, causing unnecessary alerts.

- Limited scalability: As cyber threats evolve, traditional systems cannot scale efficiently to analyze large volumes of threat intelligence data.

3.2 Proposed System

To overcome the limitations of the existing approach, the AI-Driven Cybersecurity Threat Intelligence Platform integrates machine learning, deep learning, and automation to enhance threat detection and response. Key features include:

- Real-time threat detection: AI algorithms continuously monitor network activities, identifying anomalies and potential cyber threats instantly.

- Adaptive learning: The platform updates its knowledge base using threat intelligence feeds, enabling detection of new attack patterns.

- Automated response mechanisms: AI-driven automation mitigates cyber threats with minimal human intervention, reducing response time.

- Behavioural analysis: Instead of relying solely on predefined signatures, AI assesses behavioural patterns to detect suspicious activities.

- Scalability & efficiency: Cloud-based AI processing ensures rapid threat analysis, even with large-scale data input.

4. SOFTWARE REQUIREMENT SPECIFICATIONS (SRS)

4.1 Functional Requirements

The AI-Driven Cybersecurity Threat Intelligence Platform must fulfill the following functional requirements:

Real-time Threat Detection: -The system should continuously monitor network activity to detect potential cyber threats.

Threat Classification: - AI models should categorize threats based on severity levels (low, medium, high).

Automated Response Mechanisms: -The platform should suggest or initiate countermeasures against detected threats.

User Authentication & Access Control: -Ensure secure access control for system users to prevent unauthorized access.

Threat Intelligence Reporting: -Generate actionable cybersecurity reports based on AI insights.

Integration with External Security Systems: -The system should be capable of integrating with SIEM tools, firewalls, and other cybersecurity solutions.

Logging & Audit Trail: -Maintain records of detected threats, responses, and system interactions for compliance.

4.2 Non-Functional Requirements

These requirements define the system's performance and operational characteristics:

Scalability: -The platform should be able to handle increasing amounts of threat data without performance degradation.

Reliability & Availability: - The system should operate with minimal downtime and provide accurate threat intelligence.

Security Measures: - Implement encryption and cybersecurity protocols to protect data integrity.

User-Friendly Interface: -The system should have an intuitive UI for cybersecurity professionals.

Performance Efficiency: - AI models should process threat data within seconds for real-time responses.

Regulatory Compliance: - Ensure adherence to cybersecurity laws and data protection standards (GDPR, ISO 27001).

4.3 Domain Requirements

The system operates within the cybersecurity domain, requiring the following specific considerations:

Threat Intelligence Data Sources: -The system should utilize feeds from global security databases, including malware repositories and attack patterns.

Machine Learning Algorithms: - AI techniques such as deep learning, anomaly detection, and behavioural analysis should be used.

Incident Response Framework: - The platform should align with industry-standard incident response methodologies.

Data Privacy Considerations: - Secure handling of sensitive information in compliance with cybersecurity regulations.

Cloud & On-Premise Deployment: - The platform should support flexible deployment options based on client needs.

5. SYSTEM ANALYSIS

5.1 Existing System

Traditional cybersecurity solutions rely on signature-based detection, rule-based filtering, and manual threat analysis. Organizations use intrusion detection systems (IDS), firewalls, and antivirus software to identify known cyber threats. However, these conventional systems struggle with sophisticated attacks like zero-day vulnerabilities and AI-driven cyber threats.

5.2 Limitations of Existing System

Despite their widespread use, traditional threat intelligence platforms have several shortcomings:

Static Rule-Based Detection: - Limited ability to detect new and evolving attack patterns.

Delayed Response: -Manual analysis slows down threat mitigation, increasing the risk of breaches.

High False Positives: - Conventional systems often misinterpret safe activities as security threats.

Lack of Automation: - Requires continuous human intervention for threat validation and response.

Limited Learning Capabilities: - Traditional systems cannot autonomously adapt to emerging cyber risks.

5.3 Proposed System

To address these limitations, the AI-Driven Cybersecurity Threat Intelligence Platform integrates machine learning and automated threat analysis to enhance cybersecurity measures. Key innovations include:

AI-Powered Threat Detection: - Uses deep learning to identify suspicious behaviors in real-time.

Automated Incident Response: -Reduces reliance on manual intervention and accelerates mitigation processes.

Behaviour-Based Anomaly Detection: -Identifies threats based on unusual patterns rather than predefined signatures.

Self-Learning Mechanisms: -AI models continuously improve by analyzing new threats and adapting security strategies.

5.4 Feasibility Study

A feasibility study evaluates the viability of implementing this AI-driven platform:

Technical Feasibility

- Utilizes AI technologies such as deep learning, NLP, and anomaly detection.

- Compatible with cloud-based infrastructure for scalability and efficient processing.

Economic Feasibility

- Reduces operational costs by minimizing the need for manual cybersecurity monitoring.

- Prevents financial losses caused by cyberattacks through proactive threat mitigation.

Operational Feasibility

- Enhances security teams' efficiency by automating complex threat analysis.

- Provides *user-friendly dashboards* for cybersecurity professionals to interpret AI-driven insights.

5.5 Use Case Analysis

The proposed system supports various cybersecurity use cases:

Use Case 1: Malware Detection and Prevention

Actors: - System Administrator, AI Platform

Scenario: - The AI engine continuously scans network traffic and detects malware before execution.

Use Case 2: Phishing Attack Detection

Actors: -Email Server, AI Threat Intelligence

Scenario: - AI analyses incoming emails, identifies potential phishing attacks, and quarantines suspicious messages.

Use Case 3: Automated Incident Response

Actors: -Security Team, AI Response System

Scenario: -Upon detecting a cyberattack, the system automatically applies mitigation strategies, such as isolating affected devices.

6. SYSTEM DESIGN

6.1 System Architecture

The AI-Driven Cybersecurity Threat Intelligence Platform follows a modular architecture designed for real-time threat detection and automated response. The architecture consists of the following components:

Architecture Layers:

1. Data Collection Layer – Gathers threat intelligence data from logs, network traffic, APIs, and security databases.

2. Preprocessing Layer – Cleans, filters, and normalizes raw cybersecurity data for AI model processing.

3. AI Analysis Layer– Uses deep learning and behavioural analysis models to classify threats.

4. Threat Intelligence Layer – Generates insights and predictions based on identified threats.

5. Response & Mitigation Layer – Executes security countermeasures, automated alerts, and system actions.

6.2 Modules Details

The system is divided into multiple modules to perform distinct cybersecurity tasks efficiently:

1. Data Collection Module

- Sources threat intelligence from logs, security databases, and real-time network traffic.

- Supports API integration with cybersecurity platforms.

2. Threat Analysis & Detection Module

- Uses AI algorithms to analyze patterns and detect anomalies.

- Implements supervised and unsupervised learning techniques for cyber threat classification.

3. Automated Response Module

- Suggests and executes security actions such as isolating compromised devices.

- Alerts cybersecurity personnel when high-risk threats are detected.

4. Reporting & Visualization Module

- Generates graphical security insights, dashboards, and reports for administrators.

- Provides risk assessment trends and system health metrics.

6.3 User Interface Design

The UI is designed to be user-friendly and intuitive for security professionals:

- Dashboard: -Displays real-time threat metrics, security insights, and system status.

- Alert System: - Provides notifications for high-risk security events.

- Threat Analysis Panel: - Shows detected vulnerabilities with corresponding risk levels.

- Incident Response Hub: - Allows users to trigger automated mitigation actions.

6.4 Activity Diagram

The activity diagram illustrates the system's operational workflow:

1. User logs into the platform → System authenticates access

2. Threat data is collected → Preprocessing and normalization of security logs

3. AI analyzes incoming threats → Classification into risk categories

4. Alerts & reports are generated → Security admin reviews threat intelligence

5. Automated or manual response actions → Mitigation strategies are executed

6. System learns and updates AI models → Improves detection accuracy

7. DATABASE DESIGN (TABLE STRUCTURE AND USECASE DIAGRAM)

7.1 Table Structure

The AI-Driven Cybersecurity Threat Intelligence Platform requires multiple database tables to manage various aspects of threat detection, user management, and system logging. Below are the essential tables:

1. Users Table

Stores authentication details and roles of system users.

Column Name	Data type	Description
user_id	INT (primary Key)	Unique identifier for users
name	VARCHAR(255)	Full name of the user
Email	VARCHAR(255)	Email for authentication
Password	VARCHAR(255)	Hashed password for security
Role	ENUM (Admin, Analyst)	User role within the system
Created_at	TIMESTAMP	Account creation date

2. Threat_Log Table

Stores detected security threats and their details.

Column Name	Data Type	Description
threat_ id	INT (Primary Key)	Unique identifier for the threat
dectected _ at	TIMESTAMP	Timestamp of detection
threat _type	VARCHAR(255)	Type of cyber threat (malware, phishing, etc.)
severity	ENUM (Low, Medium, High)	Threat severity level
source_ip	VARCHAR65)	IP address from where threat originated

3. AI_Analysis Table

Stores AI-generated threat intelligence insights.

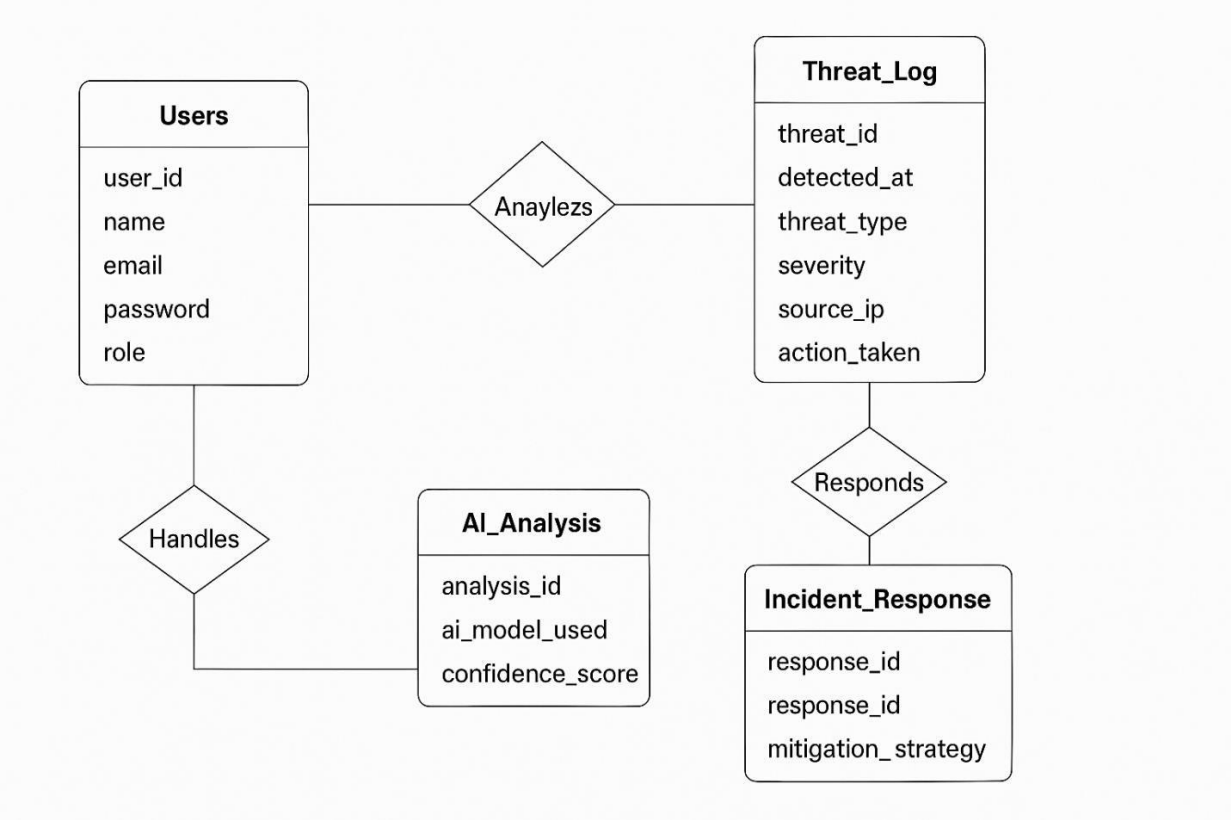
Column Name	Data type	Description
analysis-id	INT (Primary Key)	Unique identifier for analysis
threat—id	INT (Foreign Key)	References Threat_Log table
ai_model_used	VARCHAR(255)	Al model applied for analysis
confidence_score	FLOAT	Al confidence in threat detection

4. Incident_Response Table

Stores incident handling records.

Column Name	Data Type	Description
response_id	INT (Primary Key)	Unique identifier for the response
threat_id	INT (Foreign Key)	References Threat _ Log table
mitigation_strategy	VARCHAR(255)	Action taken to resolve the threat
handled_by	VARCHAR(255)	Security personnel managing the incident

7.2 UseCase Diagram



8. IMPLEMENTATION

8.1 Development of the System

The development of the AI-Driven Cybersecurity Threat Intelligence Platform follows an iterative approach, ensuring efficient threat detection and response capabilities.

Development Phases:

- Requirement Gathering:

- Collecting cybersecurity threat data from multiple sources.

- Defining AI models required for threat analysis.

- System Architecture Design:

- Structuring the data collection pipeline.

- Designing AI models for behavioural threat detection.

- AI Model Training:

- Preprocessing security logs and network traffic data.

- Training machine learning models for anomaly detection.

- System Integration:

- Connecting threat detection modules with automated response systems.

- Implementing an alert mechanism for security administrators.

- Testing & Optimization:

- Running unit, integration, and system tests to validate performance.

- Optimizing AI algorithms for accuracy improvement.

- Deployment & Monitoring:

- Deploying the system on cloud infrastructure or local servers.

- Continuous learning through threat intelligence updates.

8.2 Coding Conventions

To maintain efficiency, readability, and security in the development process, the project follows strict coding conventions:

General Coding Standards:

- Use meaningful variable names to enhance code readability.

- Apply modular programming to break down components into smaller functions.

- Ensure proper exception handling to prevent system failures.

Security Best Practices:

- Implement input validation to prevent SQL injection and other vulnerabilities.

- Use encryption techniques for sensitive cybersecurity data storage.

- Apply role-based access control to restrict unauthorized operations.

AI Model Implementation Guidelines:

- Optimize hyperparameters to improve threat detection accuracy.

- Utilize GPU acceleration for faster model training.

- Regularly update AI training data to adapt to emerging cyber threats.

9. TESTING

9.1 Testing Strategies

The testing phase ensures that the AI-Driven Cybersecurity Threat Intelligence Platform functions correctly, meets security standards, and delivers accurate threat detection. The primary testing strategies include:

- White Box Testing: Evaluates the internal logic of AI models, algorithms, and system architecture.

- Black Box Testing: Assesses the platform's functionality from an external perspective without considering its internal workings.

- Performance Testing: Measures system efficiency, response time, and scalability under high data loads.

- Security Testing: Validates the system's ability to withstand cyber threats, such as SQL injection, phishing, and denial-of-service (DoS) attacks.

8.1.1 Unit Testing

Unit testing focuses on individual components and modules to ensure they function correctly in isolation. Each module is tested independently before integration.

Key Tests:

- Threat Detection Algorithm Test: Confirms AI models correctly identify security threats based on training data.

- Data Preprocessing Test: Validates cleaning and normalization of threat intelligence inputs.

- User Authentication Test: Ensures secure login and access control mechanisms are functioning.

9.1.2 Integration Testing

Integration testing evaluates how different modules interact within the system to ensure smooth data flow and communication between components.

Key Tests:

- AI Model & Data Input Integration: Checks if the threat detection engine processes collected data accurately.

- Automated Response System Test: Validates AI-generated threat responses within the security framework.

- Database Connection Test: Ensures seamless - interactions between system components and threat logs.

9.1.3 System Testing

System testing assesses the platform's overall functionality and reliability before deployment.

Key Tests:

- End-to-End Threat Detection Test: Simulates cybersecurity attacks to confirm the AI system identifies and mitigates risks.

- Load Testing: Measures system performance when handling large volumes of threat intelligence data.

- User Interface Testing: Ensures dashboards, reports, and alerts display correct information.

10. FUTURE ENHANCEMENT AND CONCLUSION

10.1 Future Enhancement

The AI-Driven Cybersecurity Threat Intelligence Platform has great potential for continuous improvements and future expansions. Some possible enhancements include:

- Enhanced Threat Prediction Models: Incorporating advanced AI techniques such as reinforcement learning to improve cyber threat predictions.

- Integration with Blockchain Technology: Implementing blockchain-based security for enhanced data integrity and trust in threat intelligence sharing.

- Cross-Platform Threat Intelligence Sharing: Enabling secure threat data exchange between organizations to strengthen global cybersecurity efforts.

- Improved Automated Incident Response: Enhancing AI-driven automated response systems to dynamically react to new and complex cyber threats.

- AI Explainability Features: Providing transparency in AI decisions to help cybersecurity experts understand and validate threat assessments.

- Cloud-Based Scalability: Expanding cloud deployment options to improve system scalability and accessibility for enterprises.

- User Behaviour Analytics: Utilizing AI-driven user behaviour modelling for proactive - fraud and intrusion detection.

10.2 Conclusion

Cybersecurity threats are evolving rapidly, making traditional security systems increasingly ineffective. The AI-Driven Cybersecurity Threat Intelligence Platform leverages machine learning, behavioural analysis, and automation to detect and mitigate cyber threats in real time. This platform offers improved accuracy, faster response times, and adaptive learning capabilities compared to conventional systems.

Through continuous updates, scalable deployment, and AI-driven intelligence, this solution enhances cybersecurity resilience across industries. With ongoing advancements, the platform will evolve to provide even greater efficiency, security, and predictive capabilities in the fight against cybercrime.

11. BIBLIOGRAPHY

The following references were used to gather information and support the concepts discussed in the AI-Driven Cybersecurity Threat Intelligence Platform report.

Books & Research Papers:

- "Hacking: The Art of Exploitation" – Jon Erickson

- Covers network security, exploit development, and cryptography.

- "The Art of Invisibility" – Kevin Mitnick

- Explains how to stay anonymous online and protect personal data.

- "Cybersecurity for Beginners" – Raef Meeuwisse

- A great introduction to cybersecurity concepts and practices.

- "Practical Malware Analysis" – Michael Sikorski & Andrew Honig

- Focuses on malware detection and reverse engineering techniques.

- "Threat Modeling: Designing for Security" – Adam Shostack

- Discusses security design principles and risk assessment strategies.

Online Articles & Reports:

- "AI is the greatest threat—and defense—in cybersecurity today" – McKinsey

- Discusses how AI is reshaping cybersecurity, accelerating cyberattacks, and improving defense mechanisms.

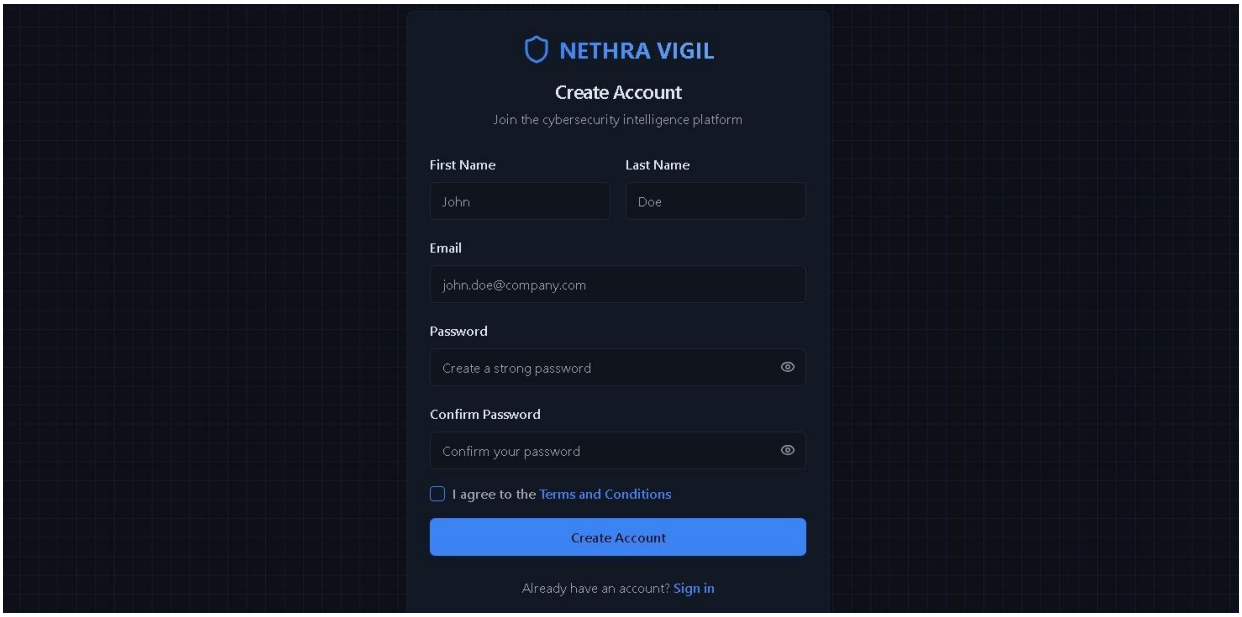
- "AI in Cybersecurity: Unlocking Smarter, Stronger Online Security" – Caltech

- Explores AI-driven security automation, threat detection, and the importance of AI in modern cybersecurity.

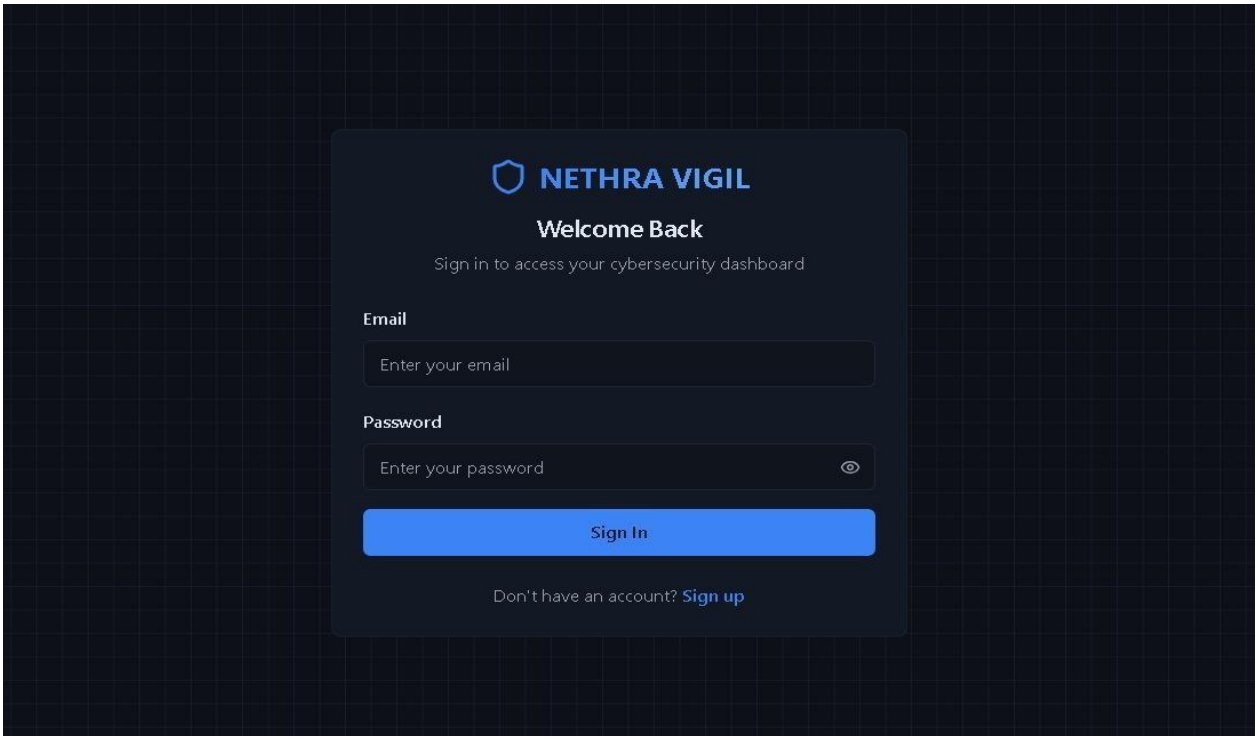
12. APPENDICES

12.1 Appendix Screenshots

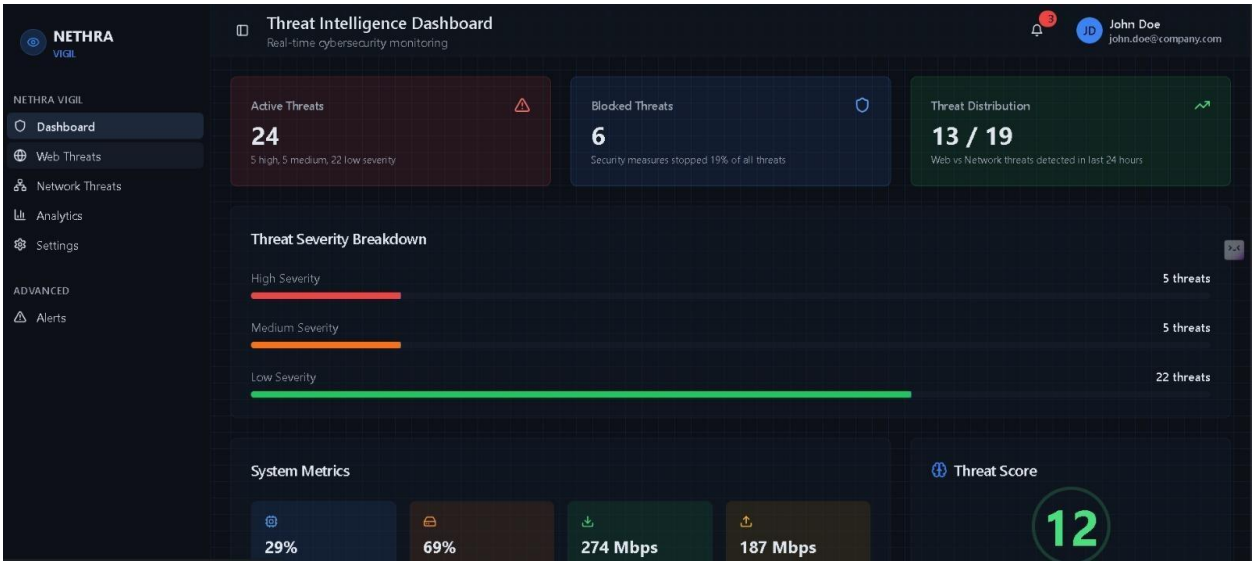
12.1.1 Signup Page



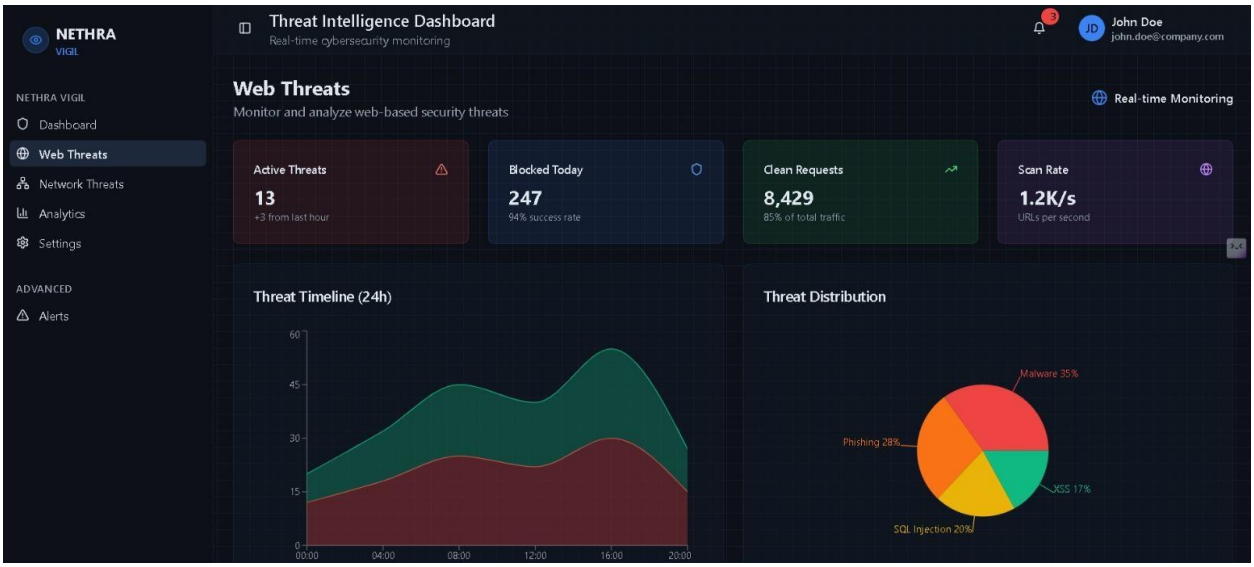
12.1.2 Login Page



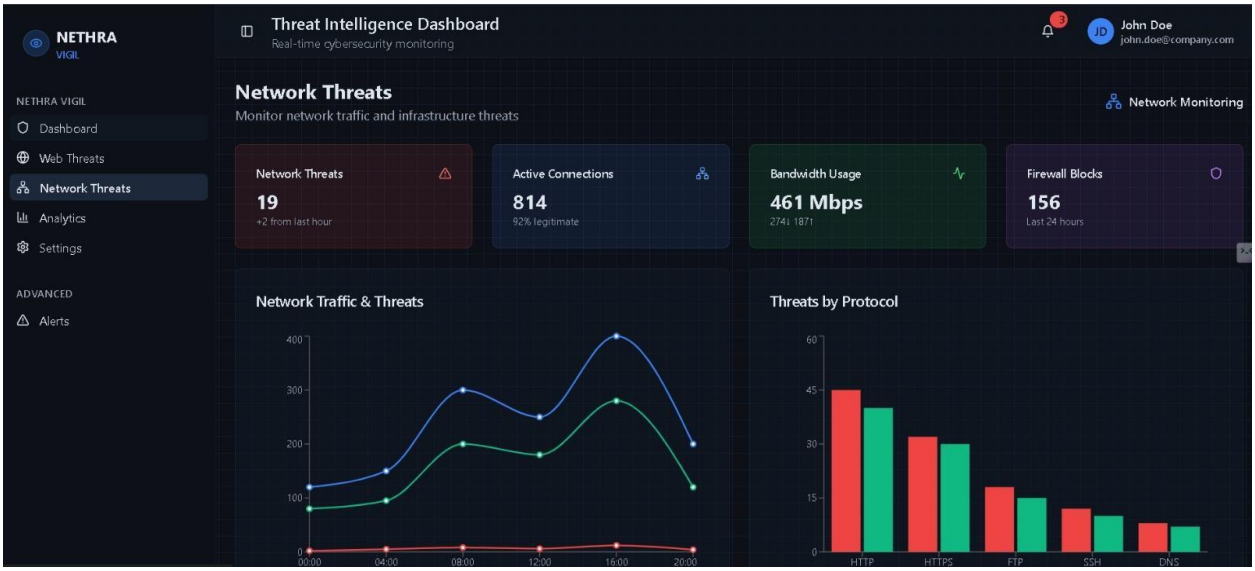
12.1.3 Dashboard



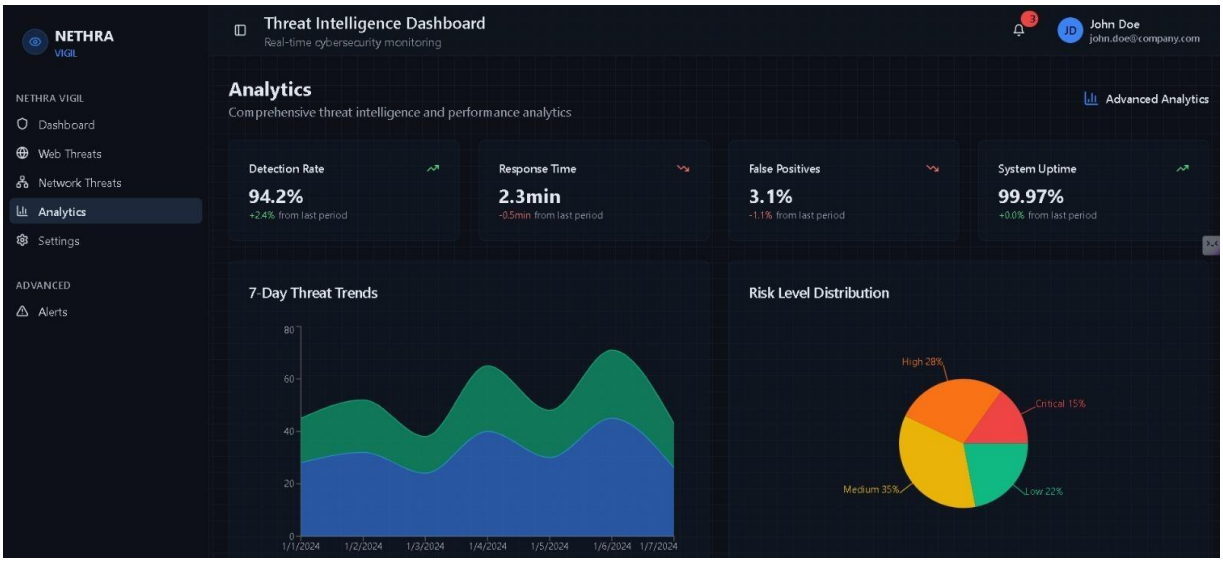
12.1.4 Web Threats



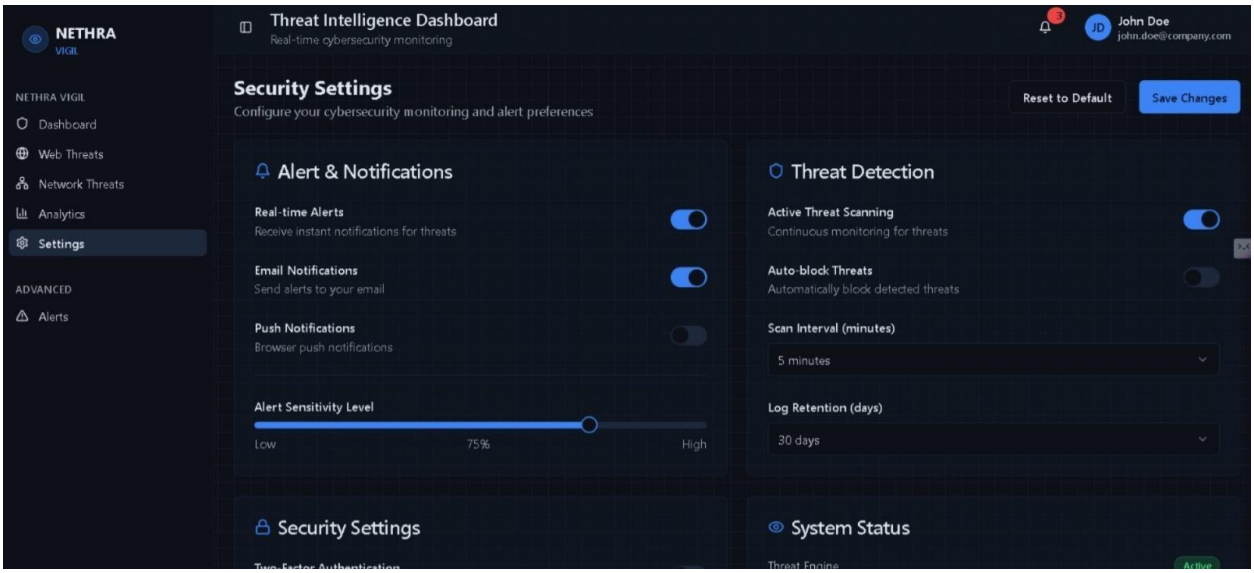
12.1.5 Network Threats



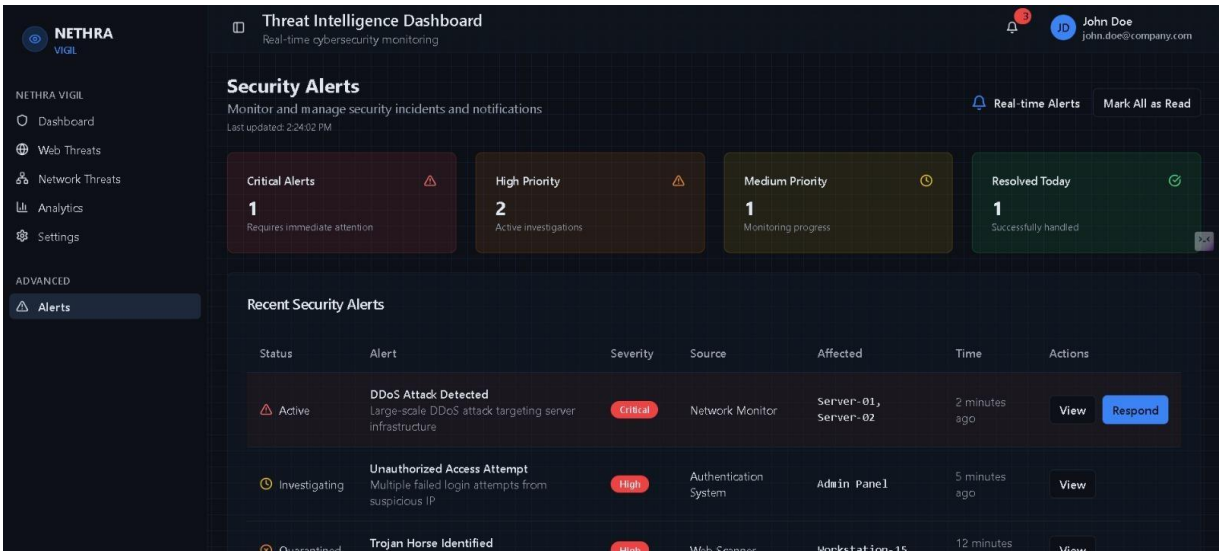
12.1.6 Analytics



12.1.7 Settings



12.1.8 Alerts



12.2 Appendix Code Snippets

12.2.1 Signup Page

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card";
import { Checkbox } from "@/components/ui/checkbox";
import { Shield, Eye, EyeOff } from "lucide-react";
import { useToast } from "@/hooks/use-toast";

const Signup = () => {
  const [formData, setFormData] = useState({
    firstName: "",
    lastName: "",
    email: "",
    password: "",
    confirmPassword: "",
  });
  const [showPassword, setShowPassword] = useState(false);
  const [showConfirmPassword, setShowConfirmPassword] = useState(false);
  const [acceptTerms, setAcceptTerms] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const navigate = useNavigate();
  const { toast } = useToast();

  const handleInputChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value,
    });
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsLoading(true);

    // Basic validation
    if (formData.password !== formData.confirmPassword) {
      toast({
        title: "Password Mismatch",
        description: "Passwords do not match",
```

```
        variant: "destructive",
      });
      setIsLoading(false);
      return;
    }

    if (!acceptTerms) {
      toast({
        title: "Terms Required",
        description: "Please accept the terms and conditions",
        variant: "destructive",
      });
      setIsLoading(false);
      return;
    }

    // Simulate account creation - replace with actual auth logic
    setTimeout(() => {
      toast({
        title: "Account Created",
        description: "Welcome to NETHRA VIGIL! Please sign in.",
      });
      navigate("/login");
      setIsLoading(false);
    }, 1000);
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-background cyber-grid py-
8">
      <Card className="w-full max-w-md mx-4 bg-card/90 backdrop-blur-sm border-border/50
glow-effect">
        <CardHeader className="text-center">
          <div className="flex items-center justify-center gap-2 mb-2">
            <Shield className="h-8 w-8 text-primary" />
            <h1 className="text-2xl font-bold bg-gradient-to-r from-primary to-blue-400 bg-clip-text
text-transparent">
              NETHRA VIGIL
            </h1>
          </div>
          <CardTitle className="text-xl text-foreground">Create Account</CardTitle>
          <CardDescription>
            Join the cybersecurity intelligence platform
          </CardDescription>
        </CardHeader>
        <CardContent>
```

```
<form onSubmit={handleSubmit} className="space-y-4">
  <div className="grid grid-cols-2 gap-4">
    <div className="space-y-2">
      <Label htmlFor="firstName">First Name</Label>
      <Input
        id="firstName"
        name="firstName"
        type="text"
        placeholder="John"
        value={formData.firstName}
        onChange={handleInputChange}
        required
        className="bg-background/50"
      />
    </div>
    <div className="space-y-2">
      <Label htmlFor="lastName">Last Name</Label>
      <Input
        id="lastName"
        name="lastName"
        type="text"
        placeholder="Doe"
        value={formData.lastName}
        onChange={handleInputChange}
        required
        className="bg-background/50"
      />
    </div>
  </div>

  <div className="space-y-2">
    <Label htmlFor="email">Email</Label>
    <Input
      id="email"
      name="email"
      type="email"
      placeholder="john.doe@company.com"
      value={formData.email}
      onChange={handleInputChange}
      required
      className="bg-background/50"
    />
  </div>

  <div className="space-y-2">
    <Label htmlFor="password">Password</Label>
```

```
            <div className="relative">
              <Input
                id="password"
                name="password"
                type={showPassword ? "text" : "password"}
                placeholder="Create a strong password"
                value={formData.password}
                onChange={handleInputChange}
                required
                className="bg-background/50 pr-10"
              />
              <Button
                type="button"
                variant="ghost"
                size="sm"
                className="absolute right-0 top-0 h-full px-3 py-2 hover:bg-transparent"
                onClick={() => setShowPassword(!showPassword)}
              >
                {showPassword ? (
                  <EyeOff className="h-4 w-4 text-muted-foreground" />
                ) : (
                  <Eye className="h-4 w-4 text-muted-foreground" />
                )}
              </Button>
            </div>
          </div>

          <div className="space-y-2">
            <Label htmlFor="confirmPassword">Confirm Password</Label>
            <div className="relative">
              <Input
                id="confirmPassword"
                name="confirmPassword"
                type={showConfirmPassword ? "text" : "password"}
                placeholder="Confirm your password"
                value={formData.confirmPassword}
                onChange={handleInputChange}
                required
                className="bg-background/50 pr-10"
              />
              <Button
                type="button"
                variant="ghost"
                size="sm"
                className="absolute right-0 top-0 h-full px-3 py-2 hover:bg-transparent"
                onClick={() => setShowConfirmPassword(!showConfirmPassword)}
```

```
              >
                {showConfirmPassword ? (
                  <EyeOff className="h-4 w-4 text-muted-foreground" />
                ) : (
                  <Eye className="h-4 w-4 text-muted-foreground" />
                )}
              </Button>
            </div>
          </div>

          <div className="flex items-center space-x-2">
            <Checkbox
              id="terms"
              checked={acceptTerms}
              onCheckedChange={(checked) => setAcceptTerms(checked as boolean)}
            />
            <Label htmlFor="terms" className="text-sm text-muted-foreground">
              I agree to the{" "}
              <span className="text-primary hover:text-primary/80 cursor-pointer">
                Terms and Conditions
              </span>
            </Label>
          </div>

          <Button
            type="submit"
            className="w-full bg-primary hover:bg-primary/90"
            disabled={isLoading}
          >
            {isLoading ? "Creating Account..." : "Create Account"}
          </Button>
        </form>

        <div className="mt-6 text-center text-sm">
          <span className="text-muted-foreground">Already have an account? </span>
          <Link to="/login" className="text-primary hover:text-primary/80 font-medium">
            Sign in
          </Link>
        </div>
      </CardContent>
    </Card>
  </div>
 );
};

export default Signup;
```

12.2.2 Login Page

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card";
import { Shield, Eye, EyeOff } from "lucide-react";
import { useToast } from "@/hooks/use-toast";

const Login = () => {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const navigate = useNavigate();
  const { toast } = useToast();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setIsLoading(true);

    // Simulate authentication - replace with actual auth logic
    setTimeout(() => {
      if (email && password) {
        toast({
          title: "Login Successful",
          description: "Welcome to NETHRA VIGIL",
        });
        navigate("/");
      } else {
        toast({
          title: "Login Failed",
          description: "Please enter valid credentials",
          variant: "destructive",
        });
      }
      setIsLoading(false);
    }, 1000);
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-background cyber-grid">
```

```
      <Card className="w-full max-w-md mx-4 bg-card/90 backdrop-blur-sm border-border/50
glow-effect">
       <CardHeader className="text-center">
        <div className="flex items-center justify-center gap-2 mb-2">
         <Shield className="h-8 w-8 text-primary" />
         <h1 className="text-2xl font-bold bg-gradient-to-r from-primary to-blue-400 bg-clip-text
text-transparent">
           NETHRA VIGIL
         </h1>
        </div>
        <CardTitle className="text-xl text-foreground">Welcome Back</CardTitle>
        <CardDescription>
         Sign in to access your cybersecurity dashboard
        </CardDescription>
       </CardHeader>
       <CardContent>
        <form onSubmit={handleSubmit} className="space-y-4">
         <div className="space-y-2">
          <Label htmlFor="email">Email</Label>
          <Input
           id="email"
           type="email"
           placeholder="Enter your email"
           value={email}
           onChange={(e) => setEmail(e.target.value)}
           required
           className="bg-background/50"
          />
         </div>

         <div className="space-y-2">
          <Label htmlFor="password">Password</Label>
          <div className="relative">
           <Input
            id="password"
            type={showPassword ? "text" : "password"}
            placeholder="Enter your password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            className="bg-background/50 pr-10"
           />
           <Button
            type="button"
            variant="ghost"
            size="sm"
```

```
                  className="absolute right-0 top-0 h-full px-3 py-2 hover:bg-transparent"
                  onClick={() => setShowPassword(!showPassword)}
                >
                  {showPassword ? (
                    <EyeOff className="h-4 w-4 text-muted-foreground" />
                  ) : (
                    <Eye className="h-4 w-4 text-muted-foreground" />
                  )}
                </Button>
              </div>
            </div>

            <Button
              type="submit"
              className="w-full bg-primary hover:bg-primary/90"
              disabled={isLoading}
            >
              {isLoading ? "Signing in..." : "Sign In"}
            </Button>
          </form>

          <div className="mt-6 text-center text-sm">
            <span className="text-muted-foreground">Don't have an account? </span>
            <Link to="/signup" className="text-primary hover:text-primary/80 font-medium">
              Sign up
            </Link>
          </div>
        </CardContent>
      </Card>
    </div>
  );
};

export default Login;
```

12.2.3 App

```
import { Toaster } from "@/components/ui/toaster";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Index from "./pages/Index";
import Login from "./pages/Login";
import Signup from "./pages/Signup";
import WebThreats from "./pages/WebThreats";
import NetworkThreats from "./pages/NetworkThreats";
import Analytics from "./pages/Analytics";
import Alerts from "./pages/Alerts";
import Settings from "./pages/Settings";
import NotFound from "./pages/NotFound";

const queryClient = new QueryClient();

const App = () => (
  <QueryClientProvider client={queryClient}>
    <TooltipProvider>
      <Toaster />
      <Sonner />
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<Index />} />
          <Route path="/login" element={<Login />} />
          <Route path="/signup" element={<Signup />} />
          <Route path="/web-threats" element={<WebThreats />} />
          <Route path="/network-threats" element={<NetworkThreats />} />
          <Route path="/analytics" element={<Analytics />} />
          <Route path="/alerts" element={<Alerts />} />
          <Route path="/settings" element={<Settings />} />
          {/* ADD ALL CUSTOM ROUTES ABOVE THE CATCH-ALL "*" ROUTE */}
          <Route path="*" element={<NotFound />} />
        </Routes>
      </BrowserRouter>
    </TooltipProvider>
  </QueryClientProvider>
);

export default App;
```

12.2.4 Dashboard

Index.tsx

```
import { DashboardLayout } from "@/components/DashboardLayout";
import { ThreatOverview } from "@/components/ThreatOverview";
import { ThreatSeverityBreakdown } from "@/components/ThreatSeverityBreakdown";
import { SystemMetrics } from "@/components/SystemMetrics";
import { ThreatScore } from "@/components/ThreatScore";
import { NetworkActivity } from "@/components/NetworkActivity";

const Index = () => {
  return (
    <DashboardLayout>
      <div className="p-6 space-y-6">
        {/* Threat Overview Cards */}
        <ThreatOverview />

        {/* Threat Severity Breakdown */}
        <ThreatSeverityBreakdown />

        <div className="grid grid-cols-1 xl:grid-cols-3 gap-6">
          {/* System Metrics */}
          <div className="xl:col-span-2">
            <SystemMetrics />
          </div>

          {/* Threat Score */}
          <div className="xl:col-span-1">
            <ThreatScore />
          </div>
        </div>

        {/* Network Activity Map */}
        <NetworkActivity />
      </div>
    </DashboardLayout>
  );
};

export default Index;
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>nethra-sentinel-sight</title>
    <meta name="description" content="Lovable Generated Project" />
    <meta name="author" content="Lovable" />

    <meta property="og:title" content="nethra-sentinel-sight" />
    <meta property="og:description" content="Lovable Generated Project" />
    <meta property="og:type" content="website" />
    <meta property="og:image" content="https://lovable.dev/opengraph-image-p98pqg.png" />

    <meta name="twitter:card" content="summary_large_image" />
    <meta name="twitter:site" content="@lovable_dev" />
    <meta name="twitter:image" content="https://lovable.dev/opengraph-image-p98pqg.png" />
  </head>

  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

12.2.4 Web Threats

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { AlertTriangle, Globe, Shield, TrendingUp } from "lucide-react";
import { ChartContainer, ChartTooltip, ChartTooltipContent } from "@/components/ui/chart";
import { AreaChart, Area, XAxis, YAxis, ResponsiveContainer, PieChart, Pie, Cell } from "recharts";
import { DashboardLayout } from "@/components/DashboardLayout";

const webThreatData = [
  { time: "00:00", threats: 12, blocked: 8 },
  { time: "04:00", threats: 18, blocked: 14 },
  { time: "08:00", threats: 25, blocked: 20 },
  { time: "12:00", threats: 22, blocked: 18 },
  { time: "16:00", threats: 30, blocked: 25 },
  { time: "20:00", threats: 15, blocked: 12 },
];

const threatTypes = [
  { name: "Malware", value: 35, color: "#ef4444" },
  { name: "Phishing", value: 28, color: "#f97316" },
  { name: "SQL Injection", value: 20, color: "#eab308" },
  { name: "XSS", value: 17, color: "#10b981" },
];

const recentThreats = [
  { id: 1, type: "Malware", url: "malicious-site.com", severity: "High", status: "Blocked", time: "2 min ago" },
  { id: 2, type: "Phishing", url: "fake-bank.net", severity: "Critical", status: "Blocked", time: "5 min ago" },
  { id: 3, type: "SQL Injection", url: "vulnerable-app.com", severity: "Medium", status: "Detected", time: "8 min ago" },
  { id: 4, type: "XSS", url: "forum-site.org", severity: "Low", status: "Monitored", time: "12 min ago" },
];

const chartConfig = {
  threats: { label: "Threats", color: "#ef4444" },
  blocked: { label: "Blocked", color: "#10b981" },
};

export default function WebThreats() {
  return (
    <DashboardLayout>
```

```
    <div className="p-6 space-y-6">
      <div className="flex items-center justify-between">
        <div>
          <h1 className="text-2xl font-bold text-foreground">Web Threats</h1>
              <p  className="text-muted-foreground">Monitor and analyze web-based security
threats</p>
        </div>
        <div className="flex items-center gap-2">
          <Globe className="h-5 w-5 text-primary" />
          <span className="text-sm font-medium">Real-time Monitoring</span>
        </div>
      </div>

      {/* Overview Cards */}
      <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
        <Card className="bg-red-500/10 border-red-500/20 threat-glow-critical">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Active Threats</CardTitle>
            <AlertTriangle className="h-4 w-4 text-red-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">13</div>
            <p className="text-xs text-muted-foreground">+3 from last hour</p>
          </CardContent>
        </Card>

        <Card className="bg-blue-500/10 border-blue-500/20 glow-effect">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Blocked Today</CardTitle>
            <Shield className="h-4 w-4 text-blue-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">247</div>
            <p className="text-xs text-muted-foreground">94% success rate</p>
          </CardContent>
        </Card>

        <Card className="bg-green-500/10 border-green-500/20 threat-glow-low">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Clean Requests</CardTitle>
            <TrendingUp className="h-4 w-4 text-green-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">8,429</div>
            <p className="text-xs text-muted-foreground">85% of total traffic</p>
          </CardContent>
```

```
        </Card>

        <Card className="bg-purple-500/10 border-purple-500/20">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Scan Rate</CardTitle>
            <Globe className="h-4 w-4 text-purple-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">1.2K/s</div>
            <p className="text-xs text-muted-foreground">URLs per second</p>
          </CardContent>
        </Card>
      </div>

      {/* Charts Section */}
      <div className="grid grid-cols-1 xl:grid-cols-2 gap-6">
        {/* Threat Timeline */}
        <Card className="bg-card/50 border border-border/50">
          <CardHeader>
            <CardTitle className="text-lg font-semibold">Threat Timeline (24h)</CardTitle>
          </CardHeader>
          <CardContent>
            <ChartContainer config={chartConfig} className="h-[300px]">
              <ResponsiveContainer width="100%" height="100%">
                <AreaChart data={webThreatData}>
                  <XAxis dataKey="time" />
                  <YAxis />
                  <ChartTooltip content={<ChartTooltipContent />} />
                  <Area
                    type="monotone"
                    dataKey="threats"
                    stackId="1"
                    stroke="#ef4444"
                    fill="#ef4444"
                    fillOpacity={0.3}
                  />
                  <Area
                    type="monotone"
                    dataKey="blocked"
                    stackId="1"
                    stroke="#10b981"
                    fill="#10b981"
                    fillOpacity={0.3}
                  />
                </AreaChart>
              </ResponsiveContainer>
```

```
            </ChartContainer>
          </CardContent>
        </Card>

        {/* Threat Distribution */}
        <Card className="bg-card/50 border border-border/50">
          <CardHeader>
            <CardTitle className="text-lg font-semibold">Threat Distribution</CardTitle>
          </CardHeader>
          <CardContent>
            <ChartContainer config={{}} className="h-[300px]">
              <ResponsiveContainer width="100%" height="100%">
                <PieChart>
                  <Pie
                    data={threatTypes}
                    cx="50%"
                    cy="50%"
                    outerRadius={80}
                    dataKey="value"
                    label={({ name, percent }) => `${name} ${(percent * 100).toFixed(0)}%`}
                  >
                    {threatTypes.map((entry, index) => (
                      <Cell key={`cell-${index}`} fill={entry.color} />
                    ))}
                  </Pie>
                  <ChartTooltip />
                </PieChart>
              </ResponsiveContainer>
            </ChartContainer>
          </CardContent>
        </Card>
      </div>

      {/* Recent Threats Table */}
      <Card className="bg-card/50 border border-border/50">
        <CardHeader>
          <CardTitle className="text-lg font-semibold">Recent Threats</CardTitle>
        </CardHeader>
        <CardContent>
          <Table>
            <TableHeader>
              <TableRow>
                <TableHead>Type</TableHead>
                <TableHead>URL</TableHead>
                <TableHead>Severity</TableHead>
                <TableHead>Status</TableHead>
```

```
              <TableHead>Time</TableHead>
            </TableRow>
          </TableHeader>
          <TableBody>
            {recentThreats.map((threat) => (
              <TableRow key={threat.id}>
                <TableCell className="font-medium">{threat.type}</TableCell>
                <TableCell className="font-mono text-sm">{threat.url}</TableCell>
                <TableCell>
                  <Badge variant={threat.severity === "Critical" ? "destructive" : threat.severity ===
"High" ? "destructive" : "secondary"}>
                    {threat.severity}
                  </Badge>
                </TableCell>
                <TableCell>
                  <Badge variant={threat.status === "Blocked" ? "default" : "outline"}>
                    {threat.status}
                  </Badge>
                </TableCell>
                <TableCell className="text-muted-foreground">{threat.time}</TableCell>
              </TableRow>
            ))}
          </TableBody>
        </Table>
      </CardContent>
    </Card>
  </div>
 </DashboardLayout>
 );
}
```

12.2.5 Network Threats

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from
"@/components/ui/table";
import { Network, Activity, Shield, AlertTriangle } from "lucide-react";
import { ChartContainer, ChartTooltip, ChartTooltipContent } from "@/components/ui/chart";
import { LineChart, Line, XAxis, YAxis, ResponsiveContainer, BarChart, Bar } from "recharts";
import { DashboardLayout } from "@/components/DashboardLayout";

const networkTrafficData = [
  { time: "00:00", inbound: 120, outbound: 80, threats: 2 },
  { time: "04:00", inbound: 150, outbound: 95, threats: 5 },
  { time: "08:00", inbound: 300, outbound: 200, threats: 8 },
  { time: "12:00", inbound: 250, outbound: 180, threats: 6 },
  { time: "16:00", inbound: 400, outbound: 280, threats: 12 },
  { time: "20:00", inbound: 200, outbound: 120, threats: 4 },
];

const protocolThreats = [
  { protocol: "HTTP", threats: 45, blocked: 40 },
  { protocol: "HTTPS", threats: 32, blocked: 30 },
  { protocol: "FTP", threats: 18, blocked: 15 },
  { protocol: "SSH", threats: 12, blocked: 10 },
  { protocol: "DNS", threats: 8, blocked: 7 },
];

const networkAlerts = [
  { id: 1, type: "DDoS Attack", source: "192.168.1.100", target: "Server-01", severity: "Critical",
status: "Mitigated", time: "1 min ago" },
  { id: 2, type: "Port Scan", source: "10.0.0.50", target: "DMZ", severity: "High", status:
"Monitoring", time: "3 min ago" },
  { id: 3, type: "Brute Force", source: "172.16.0.25", target: "SSH-Server", severity: "Medium",
status: "Blocked", time: "7 min ago" },
  { id: 4, type: "Data Exfiltration", source: "192.168.1.75", target: "Database", severity: "High",
status: "Investigating", time: "10 min ago" },
];

const chartConfig = {
  inbound: { label: "Inbound", color: "#3b82f6" },
  outbound: { label: "Outbound", color: "#10b981" },
  threats: { label: "Threats", color: "#ef4444" },
};

export default function NetworkThreats() {
  return (
```

```
    <DashboardLayout>
      <div className="p-6 space-y-6">
        <div className="flex items-center justify-between">
          <div>
            <h1 className="text-2xl font-bold text-foreground">Network Threats</h1>
            <p className="text-muted-foreground">Monitor network traffic and infrastructure
threats</p>
          </div>
          <div className="flex items-center gap-2">
            <Network className="h-5 w-5 text-primary" />
            <span className="text-sm font-medium">Network Monitoring</span>
          </div>
        </div>

        {/* Overview Cards */}
        <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
          <Card className="bg-red-500/10 border-red-500/20 threat-glow-critical">
            <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
              <CardTitle className="text-sm font-medium">Network Threats</CardTitle>
              <AlertTriangle className="h-4 w-4 text-red-400" />
            </CardHeader>
            <CardContent>
              <div className="text-2xl font-bold text-foreground">19</div>
              <p className="text-xs text-muted-foreground">+2 from last hour</p>
            </CardContent>
          </Card>

          <Card className="bg-blue-500/10 border-blue-500/20 glow-effect">
            <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
              <CardTitle className="text-sm font-medium">Active Connections</CardTitle>
              <Network className="h-4 w-4 text-blue-400" />
            </CardHeader>
            <CardContent>
              <div className="text-2xl font-bold text-foreground">814</div>
              <p className="text-xs text-muted-foreground">92% legitimate</p>
            </CardContent>
          </Card>

          <Card className="bg-green-500/10 border-green-500/20 threat-glow-low">
            <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
              <CardTitle className="text-sm font-medium">Bandwidth Usage</CardTitle>
              <Activity className="h-4 w-4 text-green-400" />
            </CardHeader>
            <CardContent>
              <div className="text-2xl font-bold text-foreground">461 Mbps</div>
              <p className="text-xs text-muted-foreground">274↓ 187↑</p>
```

```
          </CardContent>
        </Card>

        <Card className="bg-purple-500/10 border-purple-500/20">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Firewall Blocks</CardTitle>
            <Shield className="h-4 w-4 text-purple-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">156</div>
            <p className="text-xs text-muted-foreground">Last 24 hours</p>
          </CardContent>
        </Card>
      </div>

      {/* Charts Section */}
      <div className="grid grid-cols-1 xl:grid-cols-2 gap-6">
        {/* Network Traffic */}
        <Card className="bg-card/50 border border-border/50">
          <CardHeader>
            <CardTitle className="text-lg font-semibold">Network Traffic & Threats</CardTitle>
          </CardHeader>
          <CardContent>
            <ChartContainer config={chartConfig} className="h-[300px]">
              <ResponsiveContainer width="100%" height="100%">
                <LineChart data={networkTrafficData}>
                  <XAxis dataKey="time" />
                  <YAxis />
                  <ChartTooltip content={<ChartTooltipContent />} />
                  <Line type="monotone" dataKey="inbound" stroke="#3b82f6" strokeWidth={2} />
                  <Line type="monotone" dataKey="outbound" stroke="#10b981" strokeWidth={2}
/>
                  <Line type="monotone" dataKey="threats" stroke="#ef4444" strokeWidth={2} />
                </LineChart>
              </ResponsiveContainer>
            </ChartContainer>
          </CardContent>
        </Card>

        {/* Protocol Threats */}
        <Card className="bg-card/50 border border-border/50">
          <CardHeader>
            <CardTitle className="text-lg font-semibold">Threats by Protocol</CardTitle>
          </CardHeader>
          <CardContent>
            <ChartContainer config={{}} className="h-[300px]">
```

```
            <ResponsiveContainer width="100%" height="100%">
              <BarChart data={protocolThreats}>
                <XAxis dataKey="protocol" />
                <YAxis />
                <ChartTooltip />
                <Bar dataKey="threats" fill="#ef4444" />
                <Bar dataKey="blocked" fill="#10b981" />
              </BarChart>
            </ResponsiveContainer>
          </ChartContainer>
        </CardContent>
      </Card>
    </div>

    {/* Network Activity Map */}
    <Card className="bg-card/50 border border-border/50">
      <CardHeader>
        <CardTitle className="text-lg font-semibold">Live Network Topology</CardTitle>
      </CardHeader>
      <CardContent>
        <div className="relative h-64 bg-background/30 rounded-lg border border-border/30
overflow-hidden">
          <div className="absolute inset-0 flex items-center justify-center">
            <div className="grid grid-cols-8 gap-4 p-8">
              {/* Server nodes */}
              <div className="w-4 h-4 bg-blue-400 rounded-full animate-pulse border-2 border-
blue-300"></div>
              <div className="w-4 h-4 bg-green-400 rounded-full border-2 border-green-
300"></div>
              <div className="w-4 h-4 bg-green-400 rounded-full border-2 border-green-
300"></div>
              <div className="w-4 h-4 bg-green-400 rounded-full animate-pulse border-2 border-
green-300"></div>

              {/* Router nodes */}
              <div className="w-4 h-4 bg-purple-400 rounded-full border-2 border-purple-
300"></div>
              <div className="w-4 h-4 bg-purple-400 rounded-full animate-pulse border-2 border-
purple-300"></div>

              {/* Threat nodes */}
              <div className="w-4 h-4 bg-red-400 rounded-full animate-ping border-2 border-red-
300"></div>
              <div className="w-4 h-4 bg-orange-400 rounded-full animate-pulse border-2
border-orange-300"></div>
```

```
              {/* Additional nodes */}
              <div className="w-4 h-4 bg-cyan-400 rounded-full border-2 border-cyan-
300"></div>
              <div className="w-4 h-4 bg-cyan-400 rounded-full animate-pulse border-2 border-
cyan-300"></div>
              <div className="w-4 h-4 bg-cyan-400 rounded-full border-2 border-cyan-
300"></div>
              <div className="w-4 h-4 bg-yellow-400 rounded-full animate-pulse border-2
border-yellow-300"></div>
              <div className="w-4 h-4 bg-green-400 rounded-full border-2 border-green-
300"></div>
              <div className="w-4 h-4 bg-red-400 rounded-full animate-ping border-2 border-red-
300"></div>
              <div className="w-4 h-4 bg-blue-400 rounded-full border-2 border-blue-
300"></div>
              <div className="w-4 h-4 bg-purple-400 rounded-full border-2 border-purple-
300"></div>
            </div>
          </div>

          {/* Connection lines */}
          <svg className="absolute inset-0 w-full h-full">
            <defs>
              <linearGradient id="normalConnection" x1="0%" y1="0%" x2="100%"
y2="100%">
                <stop offset="0%" stopColor="rgb(59, 130, 246)" stopOpacity="0.4" />
                <stop offset="100%" stopColor="rgb(16, 185, 129)" stopOpacity="0.2" />
              </linearGradient>
              <linearGradient id="threatConnection" x1="0%" y1="0%" x2="100%" y2="100%">
                <stop offset="0%" stopColor="rgb(239, 68, 68)" stopOpacity="0.6" />
                <stop offset="100%" stopColor="rgb(251, 146, 60)" stopOpacity="0.3" />
              </linearGradient>
            </defs>
            <line x1="20%" y1="30%" x2="80%" y2="70%" stroke="url(#normalConnection)"
strokeWidth="2" />
            <line x1="30%" y1="20%" x2="70%" y2="80%" stroke="url(#threatConnection)"
strokeWidth="2" />
            <line x1="10%" y1="50%" x2="90%" y2="50%" stroke="url(#normalConnection)"
strokeWidth="2" />
            <line x1="25%" y1="60%" x2="75%" y2="40%" stroke="url(#threatConnection)"
strokeWidth="2" />
          </svg>
        </div>
      </CardContent>
    </Card>
```

```
      {/* Network Alerts Table */}
      <Card className="bg-card/50 border border-border/50">
        <CardHeader>
          <CardTitle className="text-lg font-semibold">Recent Network Alerts</CardTitle>
        </CardHeader>
        <CardContent>
          <Table>
            <TableHeader>
              <TableRow>
                <TableHead>Alert Type</TableHead>
                <TableHead>Source</TableHead>
                <TableHead>Target</TableHead>
                <TableHead>Severity</TableHead>
                <TableHead>Status</TableHead>
                <TableHead>Time</TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {networkAlerts.map((alert) => (
                <TableRow key={alert.id}>
                  <TableCell className="font-medium">{alert.type}</TableCell>
                  <TableCell className="font-mono text-sm">{alert.source}</TableCell>
                  <TableCell className="font-mono text-sm">{alert.target}</TableCell>
                  <TableCell>
                    <Badge variant={alert.severity === "Critical" ? "destructive" : alert.severity ===
"High" ? "destructive" : "secondary"}>
                      {alert.severity}
                    </Badge>
                  </TableCell>
                  <TableCell>
                    <Badge variant={alert.status === "Mitigated" || alert.status === "Blocked" ?
"default" : "outline"}>
                      {alert.status}
                    </Badge>
                  </TableCell>
                  <TableCell className="text-muted-foreground">{alert.time}</TableCell>
                </TableRow>
              ))}
            </TableBody>
          </Table>
        </CardContent>
      </Card>
    </div>
  </DashboardLayout>
 );
}
```

12.2.6 Analytics

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { BarChart3, TrendingUp, TrendingDown, Activity } from "lucide-react";
import { ChartContainer, ChartTooltip, ChartTooltipContent } from "@/components/ui/chart";
import { AreaChart, Area, XAxis, YAxis, ResponsiveContainer, BarChart, Bar, LineChart, Line,
PieChart, Pie, Cell } from "recharts";
import { DashboardLayout } from "@/components/DashboardLayout";

const threatTrendData = [
  { date: "2024-01-01", total: 45, web: 28, network: 17 },
  { date: "2024-01-02", total: 52, web: 32, network: 20 },
  { date: "2024-01-03", total: 38, web: 24, network: 14 },
  { date: "2024-01-04", total: 65, web: 40, network: 25 },
  { date: "2024-01-05", total: 48, web: 30, network: 18 },
  { date: "2024-01-06", total: 71, web: 45, network: 26 },
  { date: "2024-01-07", total: 43, web: 26, network: 17 },
];

const hourlyData = [
  { hour: "00", incidents: 8, resolved: 7 },
  { hour: "04", incidents: 12, resolved: 10 },
  { hour: "08", incidents: 25, resolved: 22 },
  { hour: "12", incidents: 18, resolved: 15 },
  { hour: "16", incidents: 32, resolved: 28 },
  { hour: "20", incidents: 15, resolved: 13 },
];

const riskCategories = [
  { name: "Critical", value: 15, color: "#ef4444" },
  { name: "High", value: 28, color: "#f97316" },
  { name: "Medium", value: 35, color: "#eab308" },
  { name: "Low", value: 22, color: "#10b981" },
];

const performanceMetrics = [
  { metric: "Detection Rate", current: 94.2, previous: 91.8, trend: "up" },
  { metric: "Response Time", current: 2.3, previous: 2.8, trend: "down", unit: "min" },
  { metric: "False Positives", current: 3.1, previous: 4.2, trend: "down", unit: "%" },
  { metric: "System Uptime", current: 99.97, previous: 99.94, trend: "up", unit: "%" },
];

const chartConfig = {
  total: { label: "Total Threats", color: "#ef4444" },
  web: { label: "Web Threats", color: "#3b82f6" },
```

```
  network: { label: "Network Threats", color: "#10b981" },
};

export default function Analytics() {
  return (
    <DashboardLayout>
      <div className="p-6 space-y-6">
        <div className="flex items-center justify-between">
          <div>
            <h1 className="text-2xl font-bold text-foreground">Analytics</h1>
            <p className="text-muted-foreground">Comprehensive threat intelligence and
performance analytics</p>
          </div>
          <div className="flex items-center gap-2">
            <BarChart3 className="h-5 w-5 text-primary" />
            <span className="text-sm font-medium">Advanced Analytics</span>
          </div>
        </div>

        {/* Performance Metrics */}
        <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
          {performanceMetrics.map((metric, index) => (
            <Card key={index} className="bg-card/50 border border-border/50">
              <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
                <CardTitle className="text-sm font-medium">{metric.metric}</CardTitle>
                {metric.trend === "up" ? (
                  <TrendingUp className="h-4 w-4 text-green-400" />
                ) : (
                  <TrendingDown className="h-4 w-4 text-red-400" />
                )}
              </CardHeader>
              <CardContent>
                <div className="text-2xl font-bold text-foreground">
                  {metric.current}{metric.unit || "%"}
                </div>
                <div className="flex items-center gap-1 text-xs">
                  <span className={metric.trend === "up" ? "text-green-400" : "text-red-400"}>
                    {metric.trend === "up" ? "+" : "-"}
                    {Math.abs(metric.current - metric.previous).toFixed(1)}
                    {metric.unit || "%"}
                  </span>
                  <span className="text-muted-foreground">from last period</span>
                </div>
              </CardContent>
            </Card>
          ))}
```

```
      </div>

      {/* Main Charts */}
      <div className="grid grid-cols-1 xl:grid-cols-2 gap-6">
       {/* Threat Trends */}
       <Card className="bg-card/50 border border-border/50">
        <CardHeader>
         <CardTitle className="text-lg font-semibold">7-Day Threat Trends</CardTitle>
        </CardHeader>
        <CardContent>
         <ChartContainer config={chartConfig} className="h-[300px]">
          <ResponsiveContainer width="100%" height="100%">
           <AreaChart data={threatTrendData}>
            <XAxis dataKey="date" tickFormatter={(value) => new
Date(value).toLocaleDateString()} />
            <YAxis />
            <ChartTooltip content={<ChartTooltipContent />} />
            <Area
             type="monotone"
             dataKey="web"
             stackId="1"
             stroke="#3b82f6"
             fill="#3b82f6"
             fillOpacity={0.6}
            />
            <Area
             type="monotone"
             dataKey="network"
             stackId="1"
             stroke="#10b981"
             fill="#10b981"
             fillOpacity={0.6}
            />
           </AreaChart>
          </ResponsiveContainer>
         </ChartContainer>
        </CardContent>
       </Card>

       {/* Risk Distribution */}
       <Card className="bg-card/50 border border-border/50">
        <CardHeader>
         <CardTitle className="text-lg font-semibold">Risk Level Distribution</CardTitle>
        </CardHeader>
        <CardContent>
         <ChartContainer config={{}} className="h-[300px]">
```

```
            <ResponsiveContainer width="100%" height="100%">
              <PieChart>
                <Pie
                  data={riskCategories}
                  cx="50%"
                  cy="50%"
                  outerRadius={80}
                  dataKey="value"
                  label={({ name, percent }) => `${name} ${(percent * 100).toFixed(0)}%`}
                >
                  {riskCategories.map((entry, index) => (
                    <Cell key={`cell-${index}`} fill={entry.color} />
                  ))}
                </Pie>
                <ChartTooltip />
              </PieChart>
            </ResponsiveContainer>
          </ChartContainer>
        </CardContent>
      </Card>
    </div>

    {/* Hourly Analysis */}
    <Card className="bg-card/50 border border-border/50">
      <CardHeader>
        <CardTitle className="text-lg font-semibold">24-Hour Incident Analysis</CardTitle>
      </CardHeader>
      <CardContent>
        <ChartContainer config={{}} className="h-[300px]">
          <ResponsiveContainer width="100%" height="100%">
            <BarChart data={hourlyData}>
              <XAxis dataKey="hour" />
              <YAxis />
              <ChartTooltip />
              <Bar dataKey="incidents" fill="#ef4444" name="Incidents" />
              <Bar dataKey="resolved" fill="#10b981" name="Resolved" />
            </BarChart>
          </ResponsiveContainer>
        </ChartContainer>
      </CardContent>
    </Card>

    {/* Summary Cards */}
    <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
      <Card className="bg-blue-500/10 border-blue-500/20 glow-effect">
        <CardHeader>
```

```
        <CardTitle className="text-lg font-semibold flex items-center gap-2">
          <Activity className="h-5 w-5 text-blue-400" />
          Threat Intelligence Summary
        </CardTitle>
      </CardHeader>
      <CardContent className="space-y-3">
        <div className="flex justify-between">
          <span className="text-sm text-muted-foreground">Most Active Hour:</span>
          <Badge variant="outline">16:00 - 17:00</Badge>
        </div>
        <div className="flex justify-between">
          <span className="text-sm text-muted-foreground">Peak Threat Day:</span>
          <Badge variant="outline">Saturday</Badge>
        </div>
        <div className="flex justify-between">
          <span className="text-sm text-muted-foreground">Common Attack Vector:</span>
          <Badge variant="destructive">Web Applications</Badge>
        </div>
      </CardContent>
    </Card>

    <Card className="bg-green-500/10 border-green-500/20 threat-glow-low">
      <CardHeader>
        <CardTitle className="text-lg font-semibold flex items-center gap-2">
          <TrendingUp className="h-5 w-5 text-green-400" />
          Security Improvements
        </CardTitle>
      </CardHeader>
      <CardContent className="space-y-3">
        <div className="flex justify-between">
          <span className="text-sm text-muted-foreground">Detection Accuracy:</span>
          <Badge variant="default">+2.4%</Badge>
        </div>
        <div className="flex justify-between">
          <span className="text-sm text-muted-foreground">Response Time:</span>
          <Badge variant="default">-18%</Badge>
        </div>
        <div className="flex justify-between">
          <span className="text-sm text-muted-foreground">False Positives:</span>
          <Badge variant="default">-26%</Badge>
        </div>
      </CardContent>
    </Card>

    <Card className="bg-purple-500/10 border-purple-500/20">
      <CardHeader>
```

```
            <CardTitle className="text-lg font-semibold flex items-center gap-2">
              <BarChart3 className="h-5 w-5 text-purple-400" />
              Predictive Insights
            </CardTitle>
          </CardHeader>
          <CardContent className="space-y-3">
            <div className="flex justify-between">
              <span className="text-sm text-muted-foreground">Risk Forecast:</span>
              <Badge variant="outline">Moderate</Badge>
            </div>
            <div className="flex justify-between">
              <span className="text-sm text-muted-foreground">Next Peak:</span>
              <Badge variant="outline">Tomorrow 4PM</Badge>
            </div>
            <div className="flex justify-between">
              <span className="text-sm text-muted-foreground">Trend:</span>
              <Badge variant="default">Improving</Badge>
            </div>
          </CardContent>
        </Card>
      </div>
    </div>
  </DashboardLayout>
 );
}
```

12.2.7 Settings

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Switch } from "@/components/ui/switch";
import { Label } from "@/components/ui/label";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@/components/ui/select";
import { Slider } from "@/components/ui/slider";
import { Separator } from "@/components/ui/separator";
import { Badge } from "@/components/ui/badge";
import { DashboardLayout } from "@/components/DashboardLayout";
import { Shield, Bell, Eye, Lock, Network, Globe, AlertTriangle, Clock } from "lucide-react";
import { useState } from "react";
import { useToast } from "@/hooks/use-toast";

export default function Settings() {
  const [settings, setSettings] = useState({
    realTimeAlerts: true,
    emailNotifications: true,
```

```
  pushNotifications: false,
  threatDetection: true,
  autoBlock: false,
  sensitivityLevel: [75],
  scanInterval: "5",
  maxLogRetention: "30",
  twoFactorAuth: false,
  sessionTimeout: "30",
  ipWhitelist: "",
  apiAccess: false
});

const { toast } = useToast();

const handleSave = () => {
  toast({
    title: "Settings Saved",
    description: "Your security settings have been updated successfully.",
  });
};

const handleReset = () => {
  setSettings({
  realTimeAlerts: true,
  emailNotifications: true,
  pushNotifications: false,
  threatDetection: true,
  autoBlock: false,
  sensitivityLevel: [75],
    scanInterval: "5",
    maxLogRetention: "30",
    twoFactorAuth: false,
    sessionTimeout: "30",
    ipWhitelist: "",
    apiAccess: false
  });
  toast({
    title: "Settings Reset",
    description: "All settings have been reset to default values.",
  });
};

return (
  <DashboardLayout>
    <div className="p-6 space-y-6">
      <div className="flex items-center justify-between">
```

```
        <div>
          <h1 className="text-2xl font-bold text-foreground">Security Settings</h1>
          <p className="text-muted-foreground">Configure your cybersecurity monitoring and
alert preferences</p>
        </div>
        <div className="flex items-center gap-4">
          <Button variant="outline" onClick={handleReset}>
            Reset to Default
          </Button>
          <Button onClick={handleSave}>
            Save Changes
          </Button>
        </div>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Alert & Notification Settings */}
        <Card className="bg-card/50 border border-border/50">
          <CardHeader>
            <CardTitle className="flex items-center gap-2">
              <Bell className="h-5 w-5 text-primary" />
              Alert & Notifications
            </CardTitle>
          </CardHeader>
          <CardContent className="space-y-6">
            <div className="flex items-center justify-between">
              <div className="space-y-0.5">
                <Label>Real-time Alerts</Label>
                <p className="text-sm text-muted-foreground">Receive instant notifications for
threats</p>
              </div>
              <Switch
                checked={settings.realTimeAlerts}
                onCheckedChange={(checked) => setSettings({...settings, realTimeAlerts:
checked})}
              />
            </div>

            <div className="flex items-center justify-between">
              <div className="space-y-0.5">
                <Label>Email Notifications</Label>
                <p className="text-sm text-muted-foreground">Send alerts to your email</p>
              </div>
              <Switch
                checked={settings.emailNotifications}
```

```
              onCheckedChange={(checked) => setSettings({...settings, emailNotifications:
checked})}
             />
            </div>

            <div className="flex items-center justify-between">
              <div className="space-y-0.5">
                <Label>Push Notifications</Label>
                <p className="text-sm text-muted-foreground">Browser push notifications</p>
              </div>
              <Switch
                checked={settings.pushNotifications}
                onCheckedChange={(checked) => setSettings({...settings, pushNotifications:
checked})}
             />
            </div>

            <Separator />

            <div className="space-y-2">
              <Label>Alert Sensitivity Level</Label>
              <Slider
                value={settings.sensitivityLevel}
                onValueChange={(value) => setSettings({...settings, sensitivityLevel: value})}
                max={100}
                step={1}
                className="w-full"
              />
              <div className="flex justify-between text-sm text-muted-foreground">
                <span>Low</span>
                <span>{settings.sensitivityLevel[0]}%</span>
                <span>High</span>
              </div>
            </div>
          </CardContent>
        </Card>

        {/* Threat Detection Settings */}
        <Card className="bg-card/50 border border-border/50">
          <CardHeader>
            <CardTitle className="flex items-center gap-2">
              <Shield className="h-5 w-5 text-primary" />
              Threat Detection
            </CardTitle>
          </CardHeader>
          <CardContent className="space-y-6">
```

```
            <div className="flex items-center justify-between">
              <div className="space-y-0.5">
                <Label>Active Threat Scanning</Label>
                <p className="text-sm text-muted-foreground">Continuous monitoring for
threats</p>
              </div>
              <Switch
                checked={settings.threatDetection}
                onCheckedChange={(checked) => setSettings({...settings, threatDetection:
checked})}
              />
            </div>

            <div className="flex items-center justify-between">
              <div className="space-y-0.5">
                <Label>Auto-block Threats</Label>
                <p className="text-sm text-muted-foreground">Automatically block detected
threats</p>
              </div>
              <Switch
                checked={settings.autoBlock}
                onCheckedChange={(checked) => setSettings({...settings, autoBlock: checked})}
              />
            </div>

            <div className="space-y-2">
              <Label>Scan Interval (minutes)</Label>
              <Select value={settings.scanInterval} onValueChange={(value) =>
setSettings({...settings, scanInterval: value})}>
                <SelectTrigger>
                  <SelectValue />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value="1">1 minute</SelectItem>
                  <SelectItem value="5">5 minutes</SelectItem>
                  <SelectItem value="15">15 minutes</SelectItem>
                  <SelectItem value="30">30 minutes</SelectItem>
                  <SelectItem value="60">1 hour</SelectItem>
                </SelectContent>
              </Select>
            </div>

            <div className="space-y-2">
              <Label>Log Retention (days)</Label>
              <Select value={settings.maxLogRetention} onValueChange={(value) =>
setSettings({...settings, maxLogRetention: value})}>
```

```
                <SelectTrigger>
                  <SelectValue />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value="7">7 days</SelectItem>
                  <SelectItem value="30">30 days</SelectItem>
                  <SelectItem value="90">90 days</SelectItem>
                  <SelectItem value="365">1 year</SelectItem>
                </SelectContent>
              </Select>
            </div>
          </CardContent>
        </Card>

        {/* Security Settings */}
        <Card className="bg-card/50 border border-border/50">
          <CardHeader>
            <CardTitle className="flex items-center gap-2">
              <Lock className="h-5 w-5 text-primary" />
              Security Settings
            </CardTitle>
          </CardHeader>
          <CardContent className="space-y-6">
            <div className="flex items-center justify-between">
              <div className="space-y-0.5">
                <Label>Two-Factor Authentication</Label>
                <p className="text-sm text-muted-foreground">Enable 2FA for enhanced
security</p>
              </div>
              <Switch
                checked={settings.twoFactorAuth}
                onCheckedChange={(checked) => setSettings({...settings, twoFactorAuth:
checked})}
              />
            </div>

            <div className="space-y-2">
              <Label>Session Timeout (minutes)</Label>
              <Select value={settings.sessionTimeout} onValueChange={(value) =>
setSettings({...settings, sessionTimeout: value})}>
                <SelectTrigger>
                  <SelectValue />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value="15">15 minutes</SelectItem>
                  <SelectItem value="30">30 minutes</SelectItem>
```

```
                <SelectItem value="60">1 hour</SelectItem>
                <SelectItem value="120">2 hours</SelectItem>
                <SelectItem value="480">8 hours</SelectItem>
              </SelectContent>
            </Select>
          </div>

          <div className="space-y-2">
            <Label>IP Whitelist</Label>
            <Input
              placeholder="192.168.1.0/24, 10.0.0.0/8"
              value={settings.ipWhitelist}
              onChange={(e) => setSettings({...settings, ipWhitelist: e.target.value})}
            />
            <p className="text-xs text-muted-foreground">Comma-separated list of allowed IP
ranges</p>
          </div>

          <div className="flex items-center justify-between">
            <div className="space-y-0.5">
              <Label>API Access</Label>
              <p className="text-sm text-muted-foreground">Allow external API
connections</p>
            </div>
            <Switch
              checked={settings.apiAccess}
              onCheckedChange={(checked) => setSettings({...settings, apiAccess: checked})}
            />
          </div>
        </CardContent>
      </Card>

      {/* System Status */}
      <Card className="bg-card/50 border border-border/50">
        <CardHeader>
          <CardTitle className="flex items-center gap-2">
            <Eye className="h-5 w-5 text-primary" />
            System Status
          </CardTitle>
        </CardHeader>
        <CardContent className="space-y-4">
          <div className="flex items-center justify-between">
            <span className="text-sm">Threat Engine</span>
            <Badge className="bg-green-500/20 text-green-400 border-green-
500/30">Active</Badge>
          </div>
```

```
            <div className="flex items-center justify-between">
              <span className="text-sm">Network Monitor</span>
              <Badge className="bg-green-500/20 text-green-400 border-green-
500/30">Running</Badge>
            </div>
            <div className="flex items-center justify-between">
              <span className="text-sm">Web Scanner</span>
              <Badge className="bg-green-500/20 text-green-400 border-green-
500/30">Online</Badge>
            </div>
            <div className="flex items-center justify-between">
              <span className="text-sm">Database</span>
              <Badge className="bg-yellow-500/20 text-yellow-400 border-yellow-
500/30">Maintenance</Badge>
            </div>
            <div className="flex items-center justify-between">
              <span className="text-sm">Last Update</span>
              <span className="text-sm text-muted-foreground">2 hours ago</span>
            </div>
          </CardContent>
        </Card>
      </div>
    </div>
  </DashboardLayout>
 );
}
```

12.2.8 Alerts

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from
"@/components/ui/table";
import { AlertTriangle, Bell, CheckCircle, XCircle, Clock } from "lucide-react";
import { DashboardLayout } from "@/components/DashboardLayout";
import { AlertModal } from "@/components/AlertModal";
import { useState, useEffect } from "react";
import { useToast } from "@/hooks/use-toast";

const initialAlerts = [
 {
  id: 1,
  type: "Critical Threat",
  title: "DDoS Attack Detected",
  description: "Large-scale DDoS attack targeting server infrastructure",
  severity: "Critical",
```

```
  status: "Active",
  time: "2 minutes ago",
  source: "Network Monitor",
  affected: "Server-01, Server-02"
 },
 {
  id: 2,
  type: "Security Breach",
  title: "Unauthorized Access Attempt",
  description: "Multiple failed login attempts from suspicious IP",
  severity: "High",
  status: "Investigating",
  time: "5 minutes ago",
  source: "Authentication System",
  affected: "Admin Panel"
 },
 {
  id: 3,
  type: "Malware Detection",
  title: "Trojan Horse Identified",
  description: "Malicious file detected in network traffic",
  severity: "High",
  status: "Quarantined",
  time: "12 minutes ago",
  source: "Web Scanner",
  affected: "Workstation-15"
 },
 {
  id: 4,
  type: "Policy Violation",
  title: "Data Transfer Anomaly",
  description: "Unusual data transfer pattern detected",
  severity: "Medium",
  status: "Monitoring",
  time: "18 minutes ago",
  source: "Data Loss Prevention",
  affected: "Database Server"
 },
 {
  id: 5,
  type: "System Alert",
  title: "High CPU Usage",
  description: "CPU usage exceeded 90% threshold",
  severity: "Low",
  status: "Resolved",
  time: "25 minutes ago",
```

```
    source: "System Monitor",
    affected: "Web Server"
  }
];

const getSeverityColor = (severity: string) => {
  switch (severity) {
    case "Critical": return "destructive";
    case "High": return "destructive";
    case "Medium": return "secondary";
    case "Low": return "outline";
    default: return "outline";
  }
};

const getStatusIcon = (status: string) => {
  switch (status) {
    case "Active": return <AlertTriangle className="h-4 w-4 text-red-400" />;
    case "Investigating": return <Clock className="h-4 w-4 text-yellow-400" />;
    case "Resolved": return <CheckCircle className="h-4 w-4 text-green-400" />;
    case "Quarantined": return <XCircle className="h-4 w-4 text-orange-400" />;
    default: return <Bell className="h-4 w-4 text-blue-400" />;
  }
};

export default function Alerts() {
  const [alerts, setAlerts] = useState(initialAlerts);
  const [selectedAlert, setSelectedAlert] = useState(null);
  const [modalMode, setModalMode] = useState<'view' | 'respond'>('view');
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [lastUpdate, setLastUpdate] = useState(new Date());
  const { toast } = useToast();

  // Simulate real-time updates
  useEffect(() => {
    const interval = setInterval(() => {
      setLastUpdate(new Date());
      // Simulate new alerts occasionally
      if (Math.random() < 0.1) {
        const newAlert = {
          id: Date.now(),
          type: "New Threat",
          title: "Suspicious Activity Detected",
          description: "Anomalous network behavior identified",
          severity: Math.random() > 0.5 ? "Medium" : "Low",
          status: "Active",
```

```
        time: "Just now",
        source: "AI Monitor",
        affected: "Network-Gateway"
      };
      setAlerts(prev => [newAlert, ...prev]);
      toast({
        title: "New Alert",
        description: newAlert.title,
        variant: "destructive",
      });
    }
  }, 30000); // Update every 30 seconds

  return () => clearInterval(interval);
}, [toast]);

const handleMarkAllAsRead = () => {
  toast({
    title: "Alerts Marked as Read",
    description: "All alerts have been marked as read.",
  });
};

const handleViewAlert = (alert: any) => {
  setSelectedAlert(alert);
  setModalMode('view');
  setIsModalOpen(true);
};

const handleRespondToAlert = (alert: any) => {
  setSelectedAlert(alert);
  setModalMode('respond');
  setIsModalOpen(true);
};

const criticalCount = alerts.filter(a => a.severity === "Critical").length;
const highCount = alerts.filter(a => a.severity === "High").length;
const mediumCount = alerts.filter(a => a.severity === "Medium").length;
const resolvedCount = alerts.filter(a => a.status === "Resolved").length;

return (
  <DashboardLayout>
    <div className="p-6 space-y-6">
      <div className="flex items-center justify-between">
        <div>
          <h1 className="text-2xl font-bold text-foreground">Security Alerts</h1>
```

```
          <p className="text-muted-foreground">Monitor and manage security incidents and
notifications</p>
          <p className="text-xs text-muted-foreground mt-1">Last updated:
{lastUpdate.toLocaleTimeString()}</p>
        </div>
        <div className="flex items-center gap-4">
          <div className="flex items-center gap-2">
            <Bell className="h-5 w-5 text-primary" />
            <span className="text-sm font-medium">Real-time Alerts</span>
          </div>
          <Button variant="outline" size="sm" onClick={handleMarkAllAsRead}>
            Mark All as Read
          </Button>
        </div>
      </div>

      {/* Alert Summary */}
      <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
        <Card className="bg-red-500/10 border-red-500/20 threat-glow-critical">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Critical Alerts</CardTitle>
            <AlertTriangle className="h-4 w-4 text-red-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">{criticalCount}</div>
            <p className="text-xs text-muted-foreground">Requires immediate attention</p>
          </CardContent>
        </Card>

        <Card className="bg-orange-500/10 border-orange-500/20 threat-glow-high">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">High Priority</CardTitle>
            <AlertTriangle className="h-4 w-4 text-orange-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">{highCount}</div>
            <p className="text-xs text-muted-foreground">Active investigations</p>
          </CardContent>
        </Card>
        <Card className="bg-yellow-500/10 border-yellow-500/20 threat-glow-medium">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Medium Priority</CardTitle>
            <Clock className="h-4 w-4 text-yellow-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">{mediumCount}</div>
```

```
            <p className="text-xs text-muted-foreground">Monitoring progress</p>
          </CardContent>
        </Card>
        <Card className="bg-green-500/10 border-green-500/20 threat-glow-low">
          <CardHeader className="flex flex-row items-center justify-between space-y-0 pb-2">
            <CardTitle className="text-sm font-medium">Resolved Today</CardTitle>
            <CheckCircle className="h-4 w-4 text-green-400" />
          </CardHeader>
          <CardContent>
            <div className="text-2xl font-bold text-foreground">{resolvedCount}</div>
            <p className="text-xs text-muted-foreground">Successfully handled</p>
          </CardContent>
        </Card>
      </div>
      {/* Alerts Table */}
      <Card className="bg-card/50 border border-border/50">
        <CardHeader>
          <CardTitle className="text-lg font-semibold">Recent Security Alerts</CardTitle>
        </CardHeader>
        <CardContent>
          <Table>
            <TableHeader>
              <TableRow>
                <TableHead>Status</TableHead>
                <TableHead>Alert</TableHead>
                <TableHead>Severity</TableHead>
                <TableHead>Source</TableHead>
                <TableHead>Affected</TableHead>
                <TableHead>Time</TableHead>
                <TableHead>Actions</TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {alerts.map((alert) => (
                <TableRow key={alert.id} className={alert.severity === "Critical" ? "bg-red-
500/5" : ""}>
                  <TableCell>
                    <div className="flex items-center gap-2">
                      {getStatusIcon(alert.status)}
                      <span className="text-sm">{alert.status}</span>
                    </div>
                  </TableCell>
                  <TableCell>
                    <div>
                      <div className="font-medium">{alert.title}</div>
                      <div className="text-sm text-muted-foreground">{alert.description}</div>
```

```
                  </div>
                </TableCell>
                <TableCell>
                  <Badge variant={getSeverityColor(alert.severity)}>
                    {alert.severity}
                  </Badge>
                </TableCell>
                <TableCell className="text-sm">{alert.source}</TableCell>
                <TableCell className="text-sm font-mono">{alert.affected}</TableCell>
                <TableCell className="text-sm text-muted-foreground">{alert.time}</TableCell>
                <TableCell>
                  <div className="flex items-center gap-2">
                    <Button
                      variant="outline"
                      size="sm"
                      onClick={() => handleViewAlert(alert)}
                    >
                      View
                    </Button>
                    {alert.status === "Active" && (
                      <Button
                        variant="default"
                        size="sm"
                        onClick={() => handleRespondToAlert(alert)}
                      >
                        Respond
                      </Button>
                    )}
                  </div>
                </TableCell>
              </TableRow>
            ))}
          </TableBody>
        </Table>
      </CardContent>
    </Card>
    <AlertModal
      alert={selectedAlert}
      isOpen={isModalOpen}
      onClose={() => setIsModalOpen(false)}
      mode={modalMode}
    />
  </div>
 </DashboardLayout>
 );
}
```